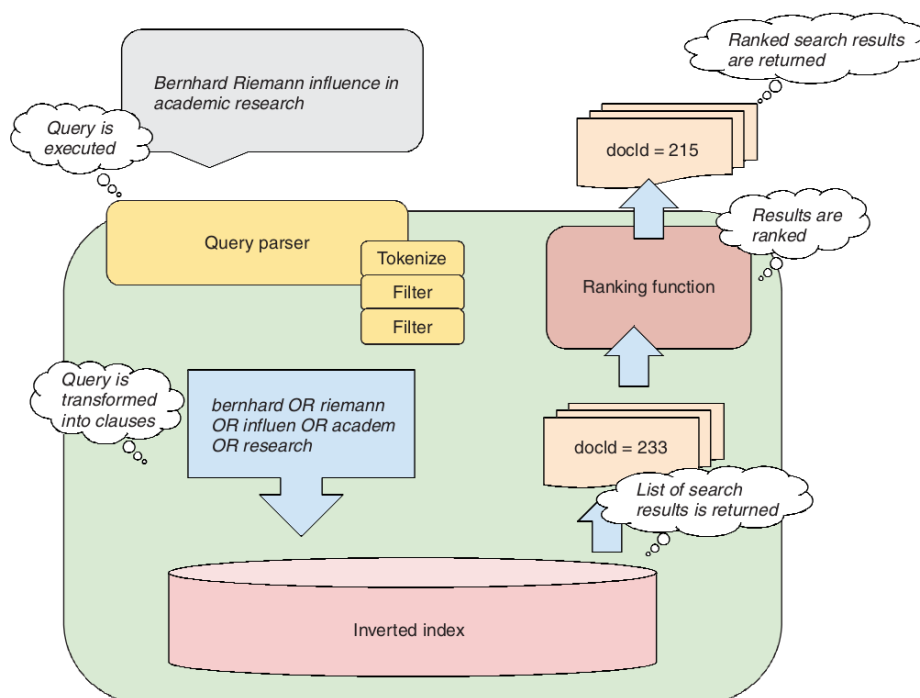


Προγραμματιστική εργασία: Σημασιολογική ανάκτηση

Ο σκοπός της εργασίας είναι να εξασκηθείτε σε κλασικές μεθόδους και μοντέλα ανάκτησης πληροφορίας, αλλά και να εφαρμόσετε state-of-the-art τεχνικές για να βελτιώσετε τα αποτελέσματα ενός συστήματος ανάκτησης πάνω σε πραγματικά δεδομένα.

Ένα μοντέλο ανάκτησης είναι ένας μαθηματικός φορμαλισμός με βάση τον οποίο πραγματοποιείται η λειτουργία της ανάκτησης. Ένα σύστημα ανάκτησης, όπως για παράδειγμα η δημοφιλής μηχανή αναζήτησης Google, υιοθετεί ένα συγκεκριμένο μοντέλο ανάκτησης και το υλοποιεί με βάση την αντίστοιχη μαθηματική θεωρία. Ένα παράδειγμα ενός συστήματος ανάκτησης παρουσιάζεται στην Εικόνα 1. Ο χρήστης αρχικά υποβάλλει ένα ερώτημα στο σύστημα. Το ερώτημα του χρήστη αναλύεται σε ένα σύνολο όρων και εκτελείται στη δομή δεδομένων του συστήματος ανάκτησης (για κάθε όρο πραγματοποιείται μια αναζήτηση σε μία δομή ανεστραμμένου ευρετηρίου όπως θα δούμε στο μάθημα). Τα κείμενα της συλλογής που ταιριάζουν συλλέγονται και διαβιβάζονται στη συνάρτηση στάθμισης/ομοιότητας (*ranking function*), η οποία υπολογίζει ένα σκορ ομοιότητας κάθε κειμένου με το ερώτημα του χρήστη. Η συνάρτηση ομοιότητας αποτελεί την καρδιά ενός μοντέλου ανάκτησης και ουσιαστικά είναι η μαθηματική θεωρία του. Στη συνέχεια, τα κείμενα ταξινομούνται/κατατάσσονται με βάση το σκορ τους σε φθίνουσα σειρά, δηλαδή από αυτό με το μεγαλύτερο σκορ ομοιότητας, που θεωρείται το πιο σχετικό, προς αυτό με το μικρότερο σκορ. Και επιστρέφονται στο χρήστη για παράδειγμα τα πρώτα 10 κείμενα. Η διαδικασία της ταξινόμησης ονομάζεται *στάθμιση (ranking)* ή *βαθμολόγηση (scoring)*.



Εικόνα 1: Υποβολή ερωτήματος, ανάκτηση και στάθμιση συναφών κειμένων σε ανεστραμμένο ευρετήριο.

Στη διάρκεια του μαθήματος θα γνωρίσουμε κλασικά μοντέλα ανάκτησης, καθένα από τα οποία ορίζει ένα διαφορετικό τρόπο (μαθηματικό φορμαλισμό όπως είπαμε) με τον οποίο υπολογίζει την ομοιότητα

των κειμένων με το ερώτημα του χρήστη, βασίζεται επομένως σε διαφορετική *συνάρτηση ομοιότητας*. Για παράδειγμα, θα δούμε στατιστικά μοντέλα, όπως το *μοντέλο διανυσματικού χώρου (Vector Space Model – VSM)*, τα οποία μετρούν την ομοιότητα ανάμεσα στο ερώτημα του χρήστη και στα κείμενα υπολογίζοντας στατιστικά για τους κοινούς όρους τους. Η απόφαση για τη στάθμιση ενός κειμένου βασίζεται στο πόσο συχνά ένας όρος εμφανίζεται σε ένα κείμενο ή/και σε όλη τη συλλογή. Επίσης, θα δούμε μοντέλα που υπολογίζουν την ομοιότητα με βάση τη θεωρία πιθανοτήτων, όπως το *γλωσσικό μοντέλο (Language Model – LM)* και το *πιθανοτικό μοντέλο (Okapi BM25)*. Στην περίπτωση αυτή, το σύστημα ανάκτησης σταθμίζει ένα κείμενο με βάση την πιθανότητα να είναι συναφές με το ερώτημα.

Τα κλασικά μοντέλα ανάκτησης «αντιλαμβάνονται» την ομοιότητα κειμένων και ερωτημάτων ως μια διαδικασία *λεξικής σύγκρισης* (term matching). Η απλοποιημένη ιδέα είναι ότι ένα κείμενο θεωρείται συναφές με ένα ερώτημα εφόσον περιέχει ακριβώς τους όρους του ερωτήματος. Η ιδέα αυτή είναι περιοριστική. Σε έναν ιδανικό κόσμο ένα σύστημα ανάκτησης θα προσπαθούσε να κατανοήσει την *πληροφοριακή ανάγκη* του χρήστη και να απαντήσει ανάλογα. Για παράδειγμα, θα μπορούσε να καταλάβει ότι όταν ο χρήστης υποβάλλει ένα ερώτημα “laptop” είναι σωστό και θεμιτό να του επιστρέψει ως συναφή, κείμενα που περιέχουν τη λέξη laptop, αλλά και κείμενα που περιέχουν τη λέξη notebook και όχι αναγκαστικά τη λέξη laptop. Κι αυτό γιατί οι λέξεις laptop και notebook είναι *σημασιολογικά* παρόμοιες, έχουν παρόμοια σημασία. Ένα τέτοιο σύστημα ανάκτησης προσπαθεί να καταλάβει τη σημασιολογία των λέξεων και να αποφασίσει αν ένα κείμενο είναι συναφές με ένα ερώτημα ή όχι.

Η δημιουργία ενός τέτοιου συστήματος ανάκτησης είναι δύσκολη. Όμως, όπως θα δείτε, η εφαρμογή τεχνικών βασισμένων σε νευρωνικά δίκτυα μπορεί να γεφυρώσει το χάσμα ανάμεσα στο ερώτημα του χρήστη και στην πραγματική πληροφοριακή του ανάγκη. Οι τεχνικές αυτές δημιουργούν νέες, σημασιολογικά πλούσιες αναπαραστάσεις των λέξεων, των κειμένων και των ερωτημάτων, ξεπερνώντας την απλή λεξική σύγκριση.

Στην εργασία αυτή, καλείστε να δημιουργήσετε ένα σύστημα ανάκτησης που θα εφαρμόζει κλασικά μοντέλα και μεθόδους ανάκτησης καθώς και μοντέλα ανάκτησης που βασίζονται σε νευρωνικά δίκτυα και συνδυασμούς αυτών, για να λύσετε ένα συγκεκριμένο πρόβλημα ανάκτησης πάνω σε μία συλλογή δεδομένων. Για το σκοπό αυτό θα χρησιμοποιήσετε την *ElasticSearch*, την *FAISS* και μοντέλα *transformers* στη συλλογή κειμένων *IR2025*, καθώς και το εργαλείο αξιολόγησης *trec_eval*. Η εργασία αποτελείται από τρεις φάσεις.

Φάση 1 – Baseline – Κλασική ανάκτηση (15 μονάδες)

Στην πρώτη φάση της εργασίας θα χρησιμοποιήσετε τη μηχανή αναζήτησης *ElasticSearch* και θα εφαρμόσετε ένα από τα κλασικά μοντέλα ανάκτησης που θα δούμε στο μάθημα (πχ. μοντέλο διανυσματικού χώρου, πιθανοτικό μοντέλο, κλπ.) στη συλλογή κειμένων *IR2025*. Η *IR2025* είναι μία συλλογή από 18.316 κείμενα. Περιλαμβάνει επίσης ένα σύνολο από ερωτήματα μαζί με τις σωστές συναφείς απαντήσεις. Το σύστημά ανάκτησης που θα δημιουργήσετε θα επιστρέφει τα πιο σχετικά κείμενα σε κάθε ερώτημα. Ακολουθήστε τα βήματα που ακολουθούν:

1. Προεπεξεργαστείτε τη συλλογή κειμένων *IR2025* (αρχείο *documents.csv*) προκειμένου να είναι σε κατάλληλη μορφή για να χρησιμοποιηθεί από τη μηχανή αναζήτησης *ElasticSearch*.

2. Δημιουργήστε ένα ευρετήριο από τη συλλογή χρησιμοποιώντας τη μηχανή αναζήτησης ElasticSearch. Επιλέξτε κατάλληλο Analyzer και συνάρτηση ομοιότητας (πχ. την BM25Similarity). Φτιάξτε τη δομή του ευρετηρίου σας όπως κρίνετε.
3. Εκτελέστε τα ερωτήματα (αρχείο queries.csv) πάνω στο ευρετήριο και συλλέξτε τις απαντήσεις της μηχανής, τα k πρώτα ανακτηθέντα κείμενα, για k = 20, 30, 50.
4. Αξιολογήστε τις απαντήσεις σας συγκρίνοντάς τις με τις σωστές απαντήσεις (αρχείο qrels.txt) χρησιμοποιώντας το εργαλείο αξιολόγησης trec_eval και τα μέτρα αξιολόγησης MAP (mean average precision) και Precision@k (ακρίβεια στα k πρώτα ανακτηθέντα κείμενα) για k = 5, 10, 15, 20.
5. Καταγράψτε τα πειράματά σας σε μια αναφορά. Περιγράψτε πώς υλοποιήσατε τα 4 παραπάνω βήματα, συμπεριλάβετε screenshots όπου θεωρείτε χρήσιμο (εγκατάσταση εργαλείων, εκτέλεση κώδικα, δημιουργία ευρετηρίου, αναζήτηση, εκτέλεση trec_eval), και φτιάξτε έναν πίνακα με τα αποτελέσματα του trec_eval για τις διάφορες τιμές του k. Συζητήστε τα αποτελέσματά σας. Δημιουργήστε ένα αρχείο pdf με την αναφορά σας. Θα υποβάλετε την αναφορά σας, τον κώδικά σας και τα αποτελέσματα του trec_eval σε ένα αρχείο zip με ονομασία αριθμός_μητρώου1_αριθμός_μητρώου2.zip (π.χ. 3200100_3200200.zip). Μη συμπεριλάβετε τη συλλογή κειμένων. Καταγράψτε τις πηγές σας. Θα βαθμολογηθείτε για την ποιότητα της αναφοράς σας. Η αναφορά θα πρέπει να έχει μέγεθος τουλάχιστον 2 σελίδες.

Φάση 2 – Σημασιολογική ανάκτηση χρησιμοποιώντας Transformers¹ και FAISS² (15 μονάδες)

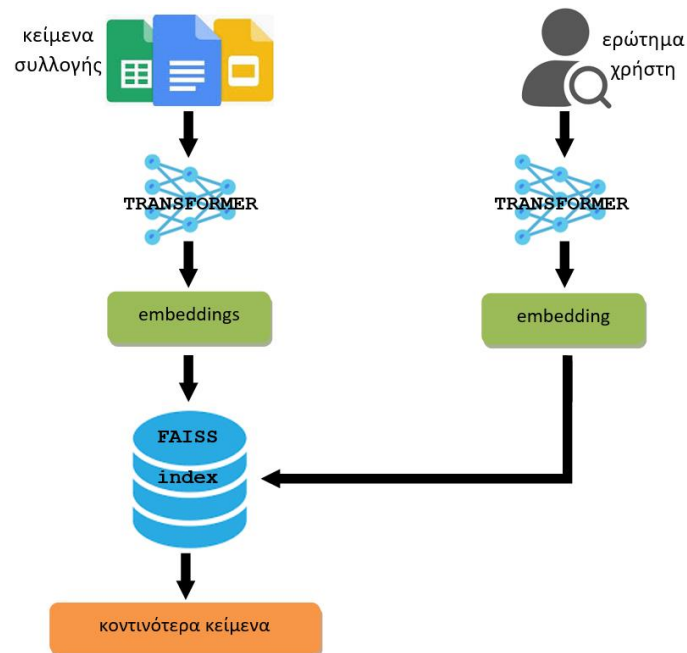
Στη φάση αυτή θα δημιουργήσετε ένα σύστημα ανάκτησης το οποίο θα βασίζεται σε σημασιολογικές αναπαραστάσεις κειμένων και ερωτημάτων παραγόμενες από μοντέλα τύπου transformers. Οι transformers αποτελούν σημαντική αρχιτεκτονική νευρωνικών δικτύων. Με τον μηχανισμό «self-attention», οι transformers μπορούν να συλλαμβάνουν σχέσεις μεταξύ λέξεων ανεξαρτήτως θέσης, δημιουργώντας πλούσιες, πυκνές διανυσματικές αναπαραστάσεις (ενθέσεις – embeddings) ολόκληρων προτάσεων ή κειμένων.

Για να αξιοποιήσουμε τις αναπαραστάσεις αυτές σε μια αποδοτική αναζήτηση, θα χρησιμοποιήσουμε τη FAISS (Facebook AI Similarity Search). Η FAISS είναι μία βιβλιοθήκη υψηλών επιδόσεων που λειτουργεί ως βάση δεδομένων διανυσμάτων (vector database): αποθηκεύει χιλιάδες ή εκατομμύρια διανύσματα σε έναν πολυδιάστατο χώρο και επιτρέπει τη γρήγορη εύρεση των πλησιέστερων γειτόνων (nearest-neighbor search) με κριτήρια όπως η συνημιτονοειδής ομοιότητα ή η ευκλείδεια απόσταση. Συνδυάζοντας τις σημασιολογικές αναπαραστάσεις των transformers με τη δυνατότητα αναζήτησης της FAISS, καλείστε να υλοποιήσετε ένα σύγχρονο σύστημα σημασιολογικής αναζήτησης (semantic search), ικανό να εντοπίζει κείμενα που σχετίζονται εννοιολογικά με το ερώτημα, ακόμη και όταν δεν μοιράζονται κοινές λέξεις.

Τα βήματα που θα πρέπει να ακολουθήσετε είναι τα παρακάτω (δείτε εικόνα 2):

1. Προεπεξεργαστείτε τη συλλογή κειμένων IR2025 (αρχείο documents.csv) προκειμένου να είναι σε κατάλληλη μορφή για να δοθεί ως είσοδος σε κάποιο μοντέλο transformers.
2. Χρησιμοποιήστε ένα προεκπαιδευμένο μοντέλο από τη βιβλιοθήκη transformers (π.χ. DistilBERT, sentence-transformers, all-MiniLM-L6-v2 ή παρόμοιο) για να μετατρέψετε κάθε κείμενο της συλλογής και κάθε ερώτημα σε πυκνά διανύσματα – ενθέσεις (embeddings). Φροντίστε τα embeddings να έχουν κοινή διάσταση (π.χ. 384 ή 768). Αποθηκεύστε τα διανύσματα για επαναχρησιμοποίηση.

3. Δημιουργήστε ένα ευρετήριο FAISS (π.χ. IndexFlatIP ή IndexFlatL2) και εισάγετε τα embeddings των κειμένων. Το FAISS παρέχει γρήγορη αναζήτηση πλησιέστερων γειτόνων (nearest-neighbor search) στον διανυσματικό χώρο.
4. Για κάθε ερώτημα: α) υπολογίστε το embedding του ερωτήματος χρησιμοποιώντας το μοντέλο transformers από το βήμα 2, β) αναζητήστε στο ευρετήριο FAISS τα k πιο κοντινά κείμενα, για $k = 20, 30, 50$, βάσει συνημιτονοειδούς ομοιότητας (cosine similarity) ή L2 απόστασης, γ) επιστρέψτε τα αποτελέσματα ταξινομημένα κατά φθίνουσα ομοιότητα.
5. Αξιολογήστε τα αποτελέσματά σας συγκρίνοντάς τα με τις σωστές απαντήσεις χρησιμοποιώντας το εργαλείο αξιολόγησης `trec_eval` και τα μέτρα αξιολόγησης MAP (mean average precision) και `Precision@k` (μέση ακρίβεια στα k πρώτα ανακτηθέντα κείμενα) για $k = 5, 10, 15, 20$.



Εικόνα 2: Σημασιολογική ανάκτηση χρησιμοποιώντας Transformers και FAISS.

6. Καταγράψτε τα πειράματά σας σε μια αναφορά όπως στην φάση 1. Περιγράψτε πώς υλοποιήσατε τα παραπάνω βήματα, συμπεριλάβετε screenshots όπου θεωρείτε χρήσιμο, και φτιάξτε έναν πίνακα με τα αποτελέσματα του `trec_eval` για τις διάφορες τιμές του k . Στην αναφορά σας συγκρίνετε τα αποτελέσματα της Φάσης 2 με τα αποτελέσματα της προηγούμενης φάσης. Υπάρχει κάποια βελτίωση; Προσπαθήστε να αιτιολογήσετε τα αποτελέσματά σας είτε αυτά είναι θετικά είτε αρνητικά.
7. Δημιουργήστε ένα αρχείο pdf με την αναφορά σας. Υποβάλετε την αναφορά σας, τον κώδικά σας και τα αποτελέσματα του `trec_eval` σε ένα αρχείο zip με ονομασία `αριθμός_μητρώου1_αριθμός_μητρώου2.zip` (πχ. `3200100_3200200.zip`). Μη συμπεριλάβετε τη συλλογή κειμένων. Καταγράψτε τις πηγές σας. Θα βαθμολογηθείτε για την ποιότητα της αναφοράς σας. Η αναφορά θα πρέπει να έχει μέγεθος τουλάχιστον 2 σελίδες.

Φάση 3 – ΠΡΟΑΙΡΕΤΙΚΗ! – Υβριδικό σύστημα ανάκτησης: συνδυασμός κλασικής και σημασιολογικής ανάκτησης (10 μονάδες)

Σε πραγματικά περιβάλλοντα αναζήτησης οι συλλογές κειμένων μπορεί να περιλαμβάνουν εκατομμύρια κείμενα. Η FAISS, αν και εξαιρετικά αποδοτική, θα χρειαζόταν σημαντικό χρόνο και μνήμη για να υπολογίσει την ομοιότητα κάθε ερωτήματος με όλα τα κείμενα. Για να αντιμετωπίσουμε το πρακτικό αυτό ζήτημα, θα συνδυάσουμε τα συστήματα των 2 προηγούμενων φάσεων, εκμεταλλευόμενοι τα πλεονεκτήματα κάθε συστήματος, όπως για παράδειγμα την ταχύτητα του ευρετηρίου της ElasticSearch και την σημασιολογική αναζήτηση που προσφέρουν οι Transformers με τη FAISS. Η βασική ιδέα είναι ότι πρώτα θα εκτελούμε ένα ερώτημα στην ElasticSearch και θα αντλούμε ένα περιορισμένο σύνολο υποψήφιων σχετικών κειμένων και έπειτα θα επαναταξινομούμε (re-ranking) τα κείμενα αυτά εκτελώντας σημασιολογική αναζήτηση με Transformers και FAISS. Έτσι, θα δημιουργήσουμε ένα υβριδικό σύστημα ανάκτησης που θα συνδυάζει την κλασική με τη σημασιολογική ανάκτηση. Αναλυτικά βήματα:

1. Για κάθε ερώτημα της συλλογής IR2025:
 - α) Εκτελέστε το ερώτημα στο ευρετήριο που υλοποιήσατε στη Φάση 1. Επιστρέψτε τα $N \gg k$ πιο σχετικά κείμενα (ενδεικτικά $N = 200$).
 - β) Με το ίδιο προεκπαιδευμένο μοντέλο transformer της Φάσης 2, δημιουργήστε ενθέσεις (embeddings) για τα N κείμενα που επέστρεψε η ElasticSearch για το ερώτημα, καθώς και για το ίδιο το ερώτημα.
 - γ) Δημιουργήστε δυναμικά ένα FAISS index με τα embeddings των N κειμένων.
 - δ) Εκτελέστε το ερώτημα. Αναζητήστε στο ευρετήριο FAISS τα k πιο κοντινά κείμενα, για $k = 20, 30, 50$, βάσει συνημιτονοειδούς ομοιότητας (cosine similarity) ή L2 απόστασης. Επιστρέψτε τα αποτελέσματα ταξινομημένα κατά φθίνουσα ομοιότητα. Η τελική κατάταξη είναι αυτή της FAISS, αλλά μπορείτε να πειραματιστείτε με συνδυασμούς βαθμολογίας (π.χ. να υπολογίσετε το σταθμισμένο μέσο όρο BM25 score και cosine similarity).
2. Αξιολογήστε τα αποτελέσματά σας συγκρίνοντάς τα με τις σωστές απαντήσεις χρησιμοποιώντας το εργαλείο αξιολόγησης trec_eval και τα μέτρα αξιολόγησης MAP (mean average precision) και Precision@k (μέση ακρίβεια στα k πρώτα ανακτηθέντα κείμενα) για $k = 5, 10, 15, 20$.
3. Καταγράψτε τα πειράματά σας σε μια αναφορά όπως στις προηγούμενες φάσεις. Περιγράψτε πώς υλοποιήσατε τα παραπάνω βήματα, τη ροή δεδομένων από ElasticSearch → Transformer → FAISS, επιλογές παραμέτρων (N , k , τύπος ευρετηρίου FAISS, μέθοδος συνδυασμού βαθμολογιών), σχόλια για πλεονεκτήματα/μειονεκτήματα της υβριδικής προσέγγισης, συμπεριλάβετε screenshots όπου θεωρείτε χρήσιμο, και φτιάξτε έναν πίνακα με τα αποτελέσματα του trec_eval για τις διάφορες τιμές του k . Στην αναφορά σας συγκρίνετε τα αποτελέσματα της Φάσης 3 με τα αποτελέσματα των προηγούμενων φάσεων. Υπάρχει κάποια βελτίωση; Προσπαθήσετε να αιτιολογήσετε τα αποτελέσματά σας είτε αυτά είναι θετικά είτε αρνητικά.
4. Δημιουργήστε ένα αρχείο pdf με την αναφορά σας. Υποβάλετε την αναφορά σας, τον κώδικά σας και τα αποτελέσματα του trec_eval σε ένα αρχείο zip με ονομασία αριθμός_μητρώου1_αριθμός_μητρώου2.zip (πχ. 3200100_3200200.zip). Μη συμπεριλάβετε τη συλλογή κειμένων. Καταγράψτε τις πηγές σας. Θα βαθμολογηθείτε για την ποιότητα της αναφοράς σας. Η αναφορά θα πρέπει να έχει μέγεθος τουλάχιστον 2 σελίδες.

Υλοποίηση

Η υλοποίηση των συστημάτων ανάκτησης *προτείνεται* να πραγματοποιηθεί με χρήση `ElasticSearch` και `Python`. Μπορείτε να δοκιμάσετε μια άλλη μηχανή αναζήτησης (πχ. `Solr`) και άλλη γλώσσα προγραμματισμού (πχ. `java`, `R`), αλλά πιθανόν να έχετε περιορισμένη υποστήριξη από τη διδάσκουσα. Δίνεται κώδικας σε `python` για τις 2 πρώτες φάσεις της εργασίας που θα παρουσιαστεί σε σχετικά εργαστήρια. Μπορείτε να γράψετε όμως δικό σας κώδικα, δική σας υλοποίηση. Το περιβάλλον υλοποίησης που θα χρησιμοποιηθεί στη διάρκεια των εργαστηρίων θα είναι το `jupyter notebook`. Η συλλογή IR2025 είναι διαθέσιμη στο `eclass`.

Βιβλιογραφία – Πηγές (διαθέσιμα στο `eclass`):

¹ Vaswani, Ashish; Shazeer et al. (December 2017). "Attention is All you Need". In 31st Conference on Neural Information Processing Systems (NIPS). Advances in Neural Information Processing Systems. Vol. 30. Curran Associates, Inc. [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)

² Matthijs Douze et al., (2024). The Faiss library. [arXiv:2401.08281](https://arxiv.org/abs/2401.08281) [cs.LG]

Εργαλεία: τα εργαλεία που θα χρειαστείτε μπορείτε να τα βρείτε στους παρακάτω συνδέσμους:

ElasticSearch	https://www.elastic.co/elasticsearch https://www.elastic.co/downloads/elasticsearch https://www.elastic.co/guide/en/elasticsearch/reference/current/getting-started.html
Python 3.x.y	https://www.python.org/downloads/
FAISS	https://github.com/facebookresearch/faiss documentation page: περιγράφει τους διαθέσιμους τύπους ευρετηρίων https://github.com/facebookresearch/faiss/wiki/Faiss-indexes
DistilBERT	πληροφορίες για το μοντέλο https://huggingface.co/docs/transformers/model_doc/distilbert https://huggingface.co/distilbert/distilbert-base-uncased
trec_eval	https://trec.nist.gov/trec_eval/ (διαθέσιμο στο <code>eclass</code> ως zip αρχείο)

Ημερομηνίες υποβολής φάσεων εργασίας (ενδέχεται να τροποποιηθούν)

Φάση 1: 3 Δεκεμβρίου 2025 (15 βαθμοί)

Φάση 2: τέλος μαθημάτων (15 βαθμοί)

Φάση 3: τέλος εξαμήνου (10 βαθμοί)

Η προγραμματιστική εργασία είναι υποχρεωτική και θα προσμετρηθεί στον τελικό βαθμό σας αν ο βαθμός γραπτής εξέτασης ≥ 4 . Πιάνει το 30% του τελικού βαθμού σας (+ bonus) Υποβάλλεται σε ομάδες των 2 φοιτητών. Κάθε φάση θα βαθμολογηθεί ξεχωριστά. Στο τέλος του εξαμήνου θα πραγματοποιηθεί **ατομική** προφορική εξέταση, από την οποία θα εξαρτηθεί ο τελικός βαθμός κάθε φοιτητή στην εργασία.